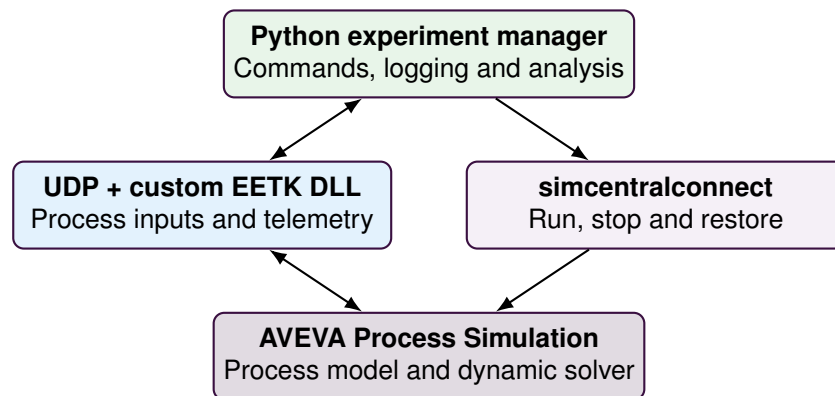

Automated Dynamic Experiments with AVEVA Process Simulation

Custom EETK DLL, UDP Communication
and Python with **simcentralconnect**

A reusable workflow for defining process variables, building the communication DLL, exchanging commands and measurements, and automating simulation runs and snapshot restoration.



Ezequiel José Valencia Urbina

Mechatronics Engineer

M.Sc. Candidate – Mechanical Systems Dynamics

University of Brasília (UnB), Brazil

`urbina.ezequiel@aluno.unb.br`

September 2026 – Revised integration workflow

Contents

1	Introduction and System Architecture	2
1.1	Responsibilities of Each Component	2
1.2	Scope and Evidence	2
2	Phase 1: Define the Variables and Communication Contract	3
2.1	Four-Tank Example	3
2.2	Recommended Protocol Extensions	3
3	Phase 2: Create or Adapt the EETK DLL	3
3.1	Start from a Compatible Project	3
3.2	Bind Inputs and Measurements	4
3.3	Separate Network Handling from Equation Evaluation	4
3.4	Telemetry and Solver Iterations	5
4	Phase 3: Configure and Validate the APS Model	5
5	Phase 4: Coordinate Python, UDP and simcentralconnect	6
5.1	Python Project Organization	6
5.2	Known Lifecycle Calls	6
5.3	Receive While the Simulator Runs	7
5.4	Distinguish Simulation Time from Wall-Clock Time	7
6	Phase 5: Execute a Repeatable Experiment	7
7	Results, Validation and Troubleshooting	8
7.1	What to Save	8
7.2	Common Problems	8
8	Extending the Workflow to Dynamic Operability	9
9	Reusable Setup Checklist and Resources	9
10	Phase 6: Using Opyrability 2.0 with the APS–UDP Interface	11
10.1	Step 1: Verify the Installed Package	11
10.2	Step 2: Define AIS, DOS and the Output Variables	11
10.3	Step 3: Implement a Deterministic Endpoint Adapter	12
10.4	Step 4: Map a Fixed-Horizon Region and Evaluate Coverage	13
10.5	Step 5: Construct the Dynamic Funnel	13
10.6	Step 6: Evaluate Boundary-Based Reduction	14
10.7	Step 7: Report Results and Connect to Closed-Loop Work	14

1 Introduction and System Architecture

This guide explains how to combine a custom UDP bridge with the official `simcentralconnect` scripting interface to automate dynamic experiments in AVEVA Process Simulation (APS). The same architecture can support tank systems, valve examples and other process models after their variables, units and operating constraints have been defined.

The custom DLL connects external commands to the process equations and exports measurements. Python uses UDP for this data exchange and `simcentralconnect` to manage the simulation lifecycle. APS remains responsible for solving the process dynamics; this workflow does not require reimplementing the process differential equations in Python.

1.1 Responsibilities of Each Component

Component	Responsibility
APS model	Equipment equations, physical constraints and dynamic state evolution.
Custom EETK DLL	Model-variable binding, command application and telemetry serialization.
UDP	Transport of commands and measurements between the DLL and Python.
<code>simcentralconnect</code>	Open the simulation, inspect its mode, run dynamics, stop the solver and restore snapshots.
Python experiment manager	Select inputs, coordinate experiments, collect results and perform analysis.

⊕ TECHNICAL NOTE

UDP does not automatically start, pause or accelerate APS. In this workflow, those execution responsibilities belong to the scripting interface and the simulator settings. UDP carries the process data. Faster-than-real-time execution is possible, but its speed depends on the model, solver configuration and computer.

1.2 Scope and Evidence

The four-tank experiment supplied with this workflow successfully combined a UDP command with `RunDynamics`, received telemetry through a simulation time of 20 seconds, and restored the base snapshot. That result demonstrates an operational communication path for the tested case. It does not establish lossless telemetry, exact command-application timing, or compatibility with every APS installation.

The DLL descriptions below are implementation guidance. The exact interface signatures, registration classes and required methods must come from the EETK project that builds against the installed APS version. No abbreviated listing in this guide should be treated as a complete, compilable DLL.

2 Phase 1: Define the Variables and Communication Contract

Before editing the DLL, identify the manipulated inputs, measured outputs, relevant states and simulation clock. Record each variable's APS path, data type, units, permitted range and direction of communication.

2.1 Four-Tank Example

JSON key	APS variable	Unit	Direction
q1	Srcs1.Q_total_1	kg/s	Python to APS
q2	Srcs1.Q_total_2	kg/s	Python to APS
h1	T1.LHeight	m	APS to Python
h2	T2.LHeight	m	APS to Python
h3	T3.LHeight	m	APS to Python
h4	T4.LHeight	m	APS to Python
sim_time	Clock binding to verify	s	APS to Python

The tested configuration uses localhost, with Python receiving on port 5005 and the DLL receiving commands on port 5006. Preserve these settings when reproducing that experiment. Assign a different port pair to each simultaneous simulation instance.

```

1 # Python -> DLL, destination 127.0.0.1:5006
2 {"q1": 0.08, "q2": 0.05}
3
4 # DLL -> Python, destination 127.0.0.1:5005
5 {"sim_time": 20.0, "h1": 0.1805, "h2": 0.1725,
6  "h3": 0.0747, "h4": 0.0782}

```

Listing 1: Existing four-tank message format: UTF-8 JSON

For a valve example, replace the input contract with the valve opening and the output contract with flow and actual opening. If an existing valve DLL accepts a plain numeric string, its Python sender must use that format. Changing to JSON requires modifying both ends.

2.2 Recommended Protocol Extensions

For repeatable automated experiments, extend the protocol with a run identifier, command identifier, telemetry sequence number and measured applied inputs. These fields are proposed additions; they were not present in the demonstrated four-tank packets. A successful UDP send only confirms local transmission, not receipt or application inside APS.

Specify how invalid commands, out-of-range values and missing updates are handled. Validate finite numeric values and apply multi-input commands atomically. Record any rejected command rather than silently changing its meaning.

3 Phase 2: Create or Adapt the EETK DLL

3.1 Start from a Compatible Project

Use the existing working DLL project or copy an official EETK example supplied for the installed APS version. Build the unchanged example first. Preserve its required interface methods, regis-

tration mechanism and lifecycle structure while adding process-specific behavior.

Check the target framework, process architecture and referenced assemblies against that working project. Installation directories and assembly names vary; use the paths actually present on the computer. A project can reside outside the SDK directory if its references and build paths are configured correctly.

3.2 Bind Inputs and Measurements

Declare the variables required by the selected process. For the four-tank example, the bridge needs two commanded inlet flows, four measured levels and access to the simulation clock. Additional signals may be needed to verify applied commands or process constraints.

A residual-based input constraint can be written as

$$r_i = u_{i,APS} - u_{i,command} = 0. \quad (1)$$

Its derivative with respect to the corresponding APS input is one when the held command is independent of the solver variables. The complete model must have a consistent specification and equation count. Release only the specification that the external equation replaces; measurement bindings do not provide additional physical closure equations.

⊕ TECHNICAL NOTE

Declare equation dependencies and derivatives according to the installed EETK contract. Adding every measured variable to every incidence row is not a general remedy for singularity. Check actual dependencies, missing equations, duplicate constraints and the toolkit's rules for exposing read-only measurements.

3.3 Separate Network Handling from Equation Evaluation

Initialize sockets once within an appropriate object lifecycle and close them when the bridge is disposed. Use nonblocking I/O or a dedicated network worker so that equation evaluation never waits for a UDP packet. UDP sockets can block unless explicitly configured otherwise.

The network worker should parse a complete command and publish it to a synchronized buffer. Model access and command acceptance must occur through supported EETK mechanisms. Hold the accepted command constant throughout the relevant numerical evaluation; changing it between residual or Jacobian evaluations can make the numerical problem inconsistent.

```

1 // Initialization:
2 //   Bind the command socket and define the telemetry destination.
3 //   Initialize a complete command and its validity information.
4 //
5 // Network worker:
6 //   Receive, parse and validate a complete input vector.
7 //   Publish it to a synchronized pending-command buffer.
8 //
9 // Supported model synchronization point:
10 //   Accept the pending command as a held input vector.
11 //
12 // Equation evaluation:
13 //   Compute residuals and derivatives using the held vector.
14 //   Do not wait for network I/O or modify model state from a worker.
15 //
```

```

16 // Telemetry:
17 //   Publish time, observations and command identity when available.
18 //
19 // Reset/disposal:
20 //   Clear stale commands; stop workers; release sockets.

```

Listing 2: Conceptual DLL design; adapt to the installed EETK interface

3.4 Telemetry and Solver Iterations

Prefer telemetry associated with accepted integration states when the interface provides such a hook. If the existing bridge transmits during residual evaluation, classify those packets as solver observations. They may include repeated timestamps, intermediate iterates or rejected trial states.

Testing `calculateDerivative == false` alone does not prove that a sample is an accepted integration step. Likewise, sending only on the last equation does not guarantee one packet per timestep. Preserve raw packets and validate the endpoint separately.

Use locale-independent serialization, explicit units and sufficient precision for the intended analysis. A value displayed to four decimal places in metres can conceal smaller level changes.

4 Phase 3: Configure and Validate the APS Model

1. Build the adapted DLL and deploy it using the add-on loading procedure for the installed APS version. Ensure the process releases the old assembly before replacing a loaded build.
2. Create or adapt the external model block. Match its variables, implementation class and bindings to the actual DLL project.
3. Connect manipulated variables and measurement aliases to the physical process. Check units and eliminate conflicting specifications.
4. Verify a consistent base solution, configure Dynamics mode, and create a named snapshot containing the intended initial condition.
5. Test a small input change. Confirm the actual manipulated variable changes and the resulting outputs respond consistently.
6. Stop and restore the snapshot. Confirm the initial conditions and clock behavior before repeating the experiment.

For the demonstrated setup, the simulation name was `Sim 3` and the snapshot was `start dym`. These are configuration values, not universal APS names.

⊕ TECHNICAL NOTE

A snapshot may restore the simulator state without restoring private DLL fields, pending UDP messages or external controller memory. Define a separate bridge-reset procedure and verify its behavior. Restoring only the four reported tank levels is not automatically equivalent to restoring the complete dynamic state.

5 Phase 4: Coordinate Python, UDP and simcentralconnect

5.1 Python Project Organization

A reusable implementation can separate responsibilities into `aveva_engine.py` for lifecycle calls, `udp_bridge.py` for communication, and `run_experiment.py` for the experiment sequence. Store configuration and result files alongside the project in clearly named directories.

Use the Python environment in which `simcentralconnect` already connects successfully. Check compatibility before changing the interpreter or installing analysis dependencies. The custom EETK DLL runs inside APS; Python communicates with it through UDP rather than loading that DLL directly.

5.2 Known Lifecycle Calls

The following excerpt uses method names observed in the supplied APS diagnostic. It illustrates lifecycle control only; the UDP acquisition worker and command synchronization must be integrated around it.

```

1 import json
2 import simcentralconnect
3
4 SIM_NAME = "Sim 3"
5 SNAPSHOT_NAME = "start dym"
6 TIMEOUT_MS = 120000
7
8 sc = simcentralconnect.connect().Result
9 sc.SetOptions(json.dumps({
10     "Timeout": TIMEOUT_MS,
11     "EnableApiLogging": "false",
12 }))
13 sm = sc.GetService("ISimulationManager")
14 snap = sc.GetService("ISnapshotManager")
15
16 def require_success(value, operation):
17     if not bool(value):
18         raise RuntimeError(f"APS operation failed: {operation}")
19
20 require_success(sm.OpenSimulation(SIM_NAME).Result, "open")
21 require_success(sm.TriggerStopSolve(
22     SIM_NAME, TIMEOUT_MS).Result, "stop before reset")
23 require_success(snap.RevertSnapshot(
24     SIM_NAME, SNAPSHOT_NAME, TIMEOUT_MS).Result, "restore")
25
26 mode = str(sm.GetSimulationMode(SIM_NAME, TIMEOUT_MS).Result)
27 if mode != "Dynamics":
28     raise RuntimeError(f"Expected Dynamics mode; received {mode}")
29
30 # Start and confirm the UDP worker before this point.
31 # Submit the input command using the agreed protocol.
32 # Check command acceptance when the bridge supports it.
33 require_success(sm.RunDynamics(
34     SIM_NAME, 20.0, "s").Result, "run dynamics")
35
36 require_success(sm.TriggerStopSolve(
37     SIM_NAME, TIMEOUT_MS).Result, "stop after run")
38 # Keep acquisition active while pending telemetry is drained.
39 # Save observations and validate the final simulator state.
40 require_success(snap.RevertSnapshot(
41     SIM_NAME, SNAPSHOT_NAME, TIMEOUT_MS).Result, "final restore")

```

Listing 3: Lifecycle excerpt based on the demonstrated APS interface

Production code must attempt stop and restoration in cleanup handling even when the experiment fails. Report cleanup failures separately and preserve the original error traceback. Shut down the acquisition worker and close sockets on every exit path.

5.3 Receive While the Simulator Runs

A blocking `.Result` call can occupy the experiment process while APS produces telemetry faster than wall-clock time. Reading the socket only after execution can overflow its buffer. A larger receive buffer alone does not guarantee complete data capture.

Use a separate acquisition process for the robust version of this workflow. That process owns the bound UDP socket, continuously receives packets, and streams them to a JSONL file. It can also send commands from the same socket through a small control channel, preserving the source-port behavior of the working test.

On Windows, protect process creation with `if __name__ == "__main__":`. Import and initialize `simcentralconnect` only in the parent process. Send small status messages through interprocess queues; do not join a worker while a large undrained queue prevents it from finishing.

5.4 Distinguish Simulation Time from Wall-Clock Time

The run argument, API timeout and telemetry clock have different meanings. The demonstrated call reached simulation time 20 seconds from the tested initial condition. Before chaining runs from nonzero times, verify whether the installed `RunDynamics` method interprets its time argument as a duration or a target time.

Read actual fields from `GetSimulationTimeDetails`; printing a .NET type name does not reveal the clock value. Record simulation times before and after each run. Use a monotonic Python timer separately to measure computational runtime.

6 Phase 5: Execute a Repeatable Experiment

1. Open the configured simulation and stop any active solver run.
2. Restore the selected snapshot and verify the initial condition and mode.
3. Reset bridge command state and remove stale telemetry from the previous experiment.
4. Start acquisition and wait until Python confirms the receive socket is ready.
5. Send the complete input vector. Verify acceptance through the implemented protocol or an appropriate actual-input measurement.
6. Advance APS using the validated time semantics while the acquisition process continues receiving.
7. Stop execution, collect pending packets and verify the final simulation time and outputs.
8. Save raw telemetry, endpoint results and experiment metadata.

9. Stop the worker, release its socket and restore the base case as appropriate for the experiment.

If commands are consumed only while the solver executes, an acknowledgement cannot necessarily arrive while APS is paused. Establish that behavior explicitly and record the actual application time; do not wait indefinitely for an unsupported paused-state acknowledgement.

7 Results, Validation and Troubleshooting

7.1 What to Save

Keep the raw JSONL telemetry and a compact result table. Each experiment should record its identifier, snapshot, requested inputs, verified applied inputs when available, initial and final simulation times, endpoint outputs, packet counts and validity status. Also retain the DLL version, Python configuration and relevant solver settings.

The supplied successful test reported 754 valid JSON packets, zero invalid packets, and timestamps from 0.1 to 20.0 seconds. Its wall-clock runtime was approximately 1.59 seconds. These are observations from one test, not a fixed sample rate or performance guarantee. Without sequence numbers, zero invalid packets does not establish zero packet loss.

7.2 Common Problems

Only a restoration message appears.

Confirm the executed file path, entry point and unbuffered console output. Print the full exception traceback before cleanup; successful restoration does not establish that the experiment ran.

The API succeeds but there are no packets.

Verify that acquisition starts before dynamics, the DLL is loaded in the selected simulation, the port pair matches, and another program has not occupied the receive port.

Inputs do not change.

Check the wire format, command validation, variable binding, specifications and actual applied input. A UDP send is not application confirmation.

Telemetry has repeated or decreasing timestamps.

Determine whether it was emitted during solver iterations, after rejected steps, or across a reset. Retain the raw evidence and separate runs before constructing trajectories.

The model becomes singular.

Inspect equation counts, released specifications, duplicate constraints, physical bindings and declared derivatives. Apply the incidence rules of the installed toolkit.

Results depend on the previous experiment.

Check snapshot completeness, DLL-held commands, pending socket messages and controller state. Verify the initial state before each independent run.

8 Extending the Workflow to Dynamic Operability

The integration layer provides a way to evaluate the APS process model from Python. It does not, by itself, implement the complete operability algorithm or an Opyrability package interface.

For a fixed initial state, running several constant input pairs for 20 seconds produces sampled endpoint outputs under constant commands. Mapping all admissible input sequences over successive intervals is a broader reachability problem.

For a discrete model, write

$$x_{k+1} = \Phi_{\Delta t}(x_k, u_k), \quad y_k = g(x_k). \quad (2)$$

Each branch must start from its own complete retained state. Reverting to the same base snapshot for every branch repeats independent experiments rather than propagating the current reachable set.

A future adapter should restore a branch state, apply an input vector, advance by a verified interval, return the output, and preserve a reproducible reference to the final state. Snapshot creation and restoration are possible building blocks, but their completeness and interaction with DLL state must be tested.

Boundary selection in a two-output projection is an approximation for a system with additional states. Two states can have the same plotted outputs and different future responses. Compare any boundary-pruning method against a small exhaustive calculation before relying on it.

Use the professor's required Opyrability version for the analysis layer and verify its installed function signatures. Match the desired output set to the actual output variables and units. Distinguish sampled points, an estimated geometric region and any calculated operability index.

9 Reusable Setup Checklist and Resources

Done	Check
☐	Inputs, outputs, states, clock, units and bounds are documented.
☐	The DLL builds against the installed APS/EETK environment.
☐	Equations, specifications and process-variable bindings are consistent.
☐	Command format, ports and reset behavior agree on both ends.
☐	Acquisition starts before execution and runs independently.
☐	Run timing and endpoint validation have been tested.
☐	Failures retain their traceback and cleanup restores the intended case.
☐	Raw data and experiment metadata are saved for reproducibility.

Project repository supplied by the author:

<https://github.com/VzequyU/AVEVA-Process-Simulation-UDP>

Tutorial link retained from the supplied template:

<https://youtu.be/r88qSzDYIMg>

Contact: urbina.ezequiel@aluno.unb.br

⊕ TECHNICAL NOTE

This revision is based on the supplied template, communication examples and successful APS execution log. Repository links identify the project; they do not certify that its current contents or every installation-specific detail have been independently verified. Consult the installed EETK and scripting examples for exact interface contracts.

10 Phase 6: Using Opyrability 2.0 with the APS–UDP Interface

This chapter describes how to place an operability analysis layer above the existing APS integration. APS evaluates the physical process, the custom DLL and UDP exchange process data, and `simcentralconnect` coordinates execution. Opyrability is used to construct and evaluate output regions through a suitable Python model adapter.

The target is the Opyrability 2.0 distribution required for the research project. The examples below reuse `multimodel_rep` and `OI_eval` from the supplied working script; their compatibility with the required distribution must be checked locally. The online documentation could not be verified during this revision. No dedicated dynamic-funnel API is assumed.

10.1 Step 1: Verify the Installed Package

Run the following diagnostic in the same Python environment used for the analysis. Save its output with the experiment configuration.

```

1 import inspect
2 from importlib import metadata
3 import opyrability
4
5 print("Imported module:", opyrability.__file__)
6 try:
7     print("Distribution version:", metadata.version("opyrability"))
8 except metadata.PackageNotFoundError:
9     print("No distribution metadata: record the source checkout.")
10
11 for name in ("multimodel_rep", "OI_eval"):
12     function = getattr(opyrability, name, None)
13     if function is None:
14         raise RuntimeError(f"Required function is unavailable: {name}")
15     print(name, inspect.signature(function))
16     print(inspect.getdoc(function))

```

Listing 4: Check package location, installed metadata and function signatures

If the professor supplied a repository checkout or a specific revision, record its commit identifier and use its examples to establish the expected API. Do not assume that an arbitrary package installation corresponds to that revision. Resolve dependency compatibility before starting a long APS experiment.

10.2 Step 2: Define AIS, DOS and the Output Variables

The Available Input Set (AIS) describes permitted manipulated inputs. The Desired Output Set (DOS) describes output requirements. The Achievable Output Set (AOS) depends on the process model, initial condition, time horizon and allowed input trajectories.

For the four-tank example, a two-input, two-output study can use

$$u = (q_1, q_2), \quad y = (h_1, h_2). \quad (3)$$

The remaining levels must still be retained when evaluating or restoring the dynamic state. Choosing two outputs for plotting does not reduce the physical process to two states.

```

1 import json
2 import numpy as np
3

```

```

4 # Example range already exercised in the four-tank experiments.
5 # Validate this range against the current model constraints.
6 AIS_BOUNDS = np.array([[0.05, 0.11], [0.05, 0.11]], dtype=float)
7 RESOLUTION = [3, 3]
8 OUTPUT_KEYS = ("h1", "h2")
9
10 # This file must contain the project's actual level requirements:
11 # {"h1": [lower_limit_m, upper_limit_m],
12 #  "h2": [lower_limit_m, upper_limit_m]}
13 with open("dos_levels_m.json", encoding="utf-8") as stream:
14     limits = json.load(stream)
15 DOS_BOUNDS = np.array([limits[key] for key in OUTPUT_KEYS], dtype=float)
16 if DOS_BOUNDS.shape != (2, 2) or not np.isfinite(DOS_BOUNDS).all():
17     raise ValueError("DOS must contain two finite [lower, upper] pairs.")
18 if np.any(DOS_BOUNDS[:, 1] <= DOS_BOUNDS[:, 0]):
19     raise ValueError("Each DOS interval must have positive width.")

```

Listing 5: Example input grid and externally supplied output requirements

The previous stationary script used discharge flows XV3.W and XV4.W, in kg/s. Its desired flow limits cannot be reused as level limits in metres. A new choice of outputs requires a matching DOS and explicitly named axes.

10.3 Step 3: Implement a Deterministic Endpoint Adapter

Begin with a fixed-horizon map that is straightforward to reproduce:

$$M_{\tau}(u) = g(\Phi_{\tau}(x_0, u)), \quad (4)$$

where every evaluation starts from the same verified state x_0 and holds u constant over the interval. The adapter must return a finite output vector in the declared order and units.

The lifecycle and UDP implementation should be encapsulated in an experiment runner. The following runner interface is a project design, not a built-in Opyrability or APS function.

```

1 def make_endpoint_model(runner, horizon_s, output_keys):
2     def model(u):
3         u = np.asarray(u, dtype=float).reshape(-1)
4         if u.shape != (2,) or not np.isfinite(u).all():
5             raise ValueError("Expected two finite input values.")
6         if np.any(u < AIS_BOUNDS[:, 0]) or np.any(u > AIS_BOUNDS[:, 1]):
7             raise ValueError("Input outside the declared AIS.")
8
9         # Project-defined method, following the earlier workflow:
10        # stop -> restore x0 -> reset bridge -> start receiver ->
11        # apply u -> advance -> validate endpoint -> save -> cleanup.
12        record = runner.evaluate_from_base(
13            inputs=u.tolist(), horizon_s=float(horizon_s)
14        )
15        if not record["valid"]:
16            raise RuntimeError(record["failure_reason"])
17        y = np.array([record["endpoint"][key] for key in output_keys])
18        if not np.isfinite(y).all():
19            raise RuntimeError("Non-finite endpoint output.")
20        return y
21    return model

```

Listing 6: Adapter contract; runner methods must be implemented in the project

The runner must establish validity from the actual endpoint time, command application, state restoration and any required process constraints. Selecting the last UDP packet without checking

its meaning is insufficient. A failed run must not be replaced with zero outputs, because that would distort the achievable region.

Evaluate APS sequentially while a single simulation instance is shared. Parallel model calls that change the same simulation, snapshot or socket state can contaminate one another.

10.4 Step 4: Map a Fixed-Horizon Region and Evaluate Coverage

After validating the runner and checking the installed function signatures, the call pattern from the existing script becomes:

```

1 from opyrability import multimodel_rep, OI_eval
2
3 # runner is the validated project-specific APS/UDP experiment runner.
4 M_20s = make_endpoint_model(runner, 20.0, OUTPUT_KEYS)
5
6 region_20s = multimodel_rep(
7     M_20s, AIS_BOUNDS, RESOLUTION, plot=False
8 )
9 oi_raw = float(OI_eval(region_20s, DOS_BOUNDS))
10 if not np.isfinite(oi_raw):
11     raise RuntimeError("Non-finite operability result.")
12 print("OI returned by the installed package:", oi_raw)

```

Listing 7: Call pattern inherited from the supplied Opyrability script

Check the package's return convention before labeling the result as a fraction or a percentage. The conceptual percentage coverage for a full-dimensional DOS is

$$OI(\tau) = 100 \frac{\mu(AOS(\tau) \cap DOS)}{\mu(DOS)}, \quad \mu(DOS) > 0, \quad (5)$$

where μ is area for two selected outputs. Record the actual region-construction method used by the installed package; an interpolated or polytopic region is an estimate based on model evaluations.

A 3×3 grid provides a useful communication and workflow check, but it does not establish geometric accuracy. Inspect the output points and refine the grid to assess changes in the region and index. If outputs are collinear, do not invent an area to satisfy a two-dimensional routine.

⊕ TECHNICAL NOTE

Label this first result as a **fixed-horizon, constant-input operability estimate**. Repeating it at several horizons gives a family of endpoint maps. It does not yet reproduce the article's enumeration of changing input sequences.

10.5 Step 5: Construct the Dynamic Funnel

To reproduce the forward-mapping logic discussed in the article, propagate retained states over successive intervals. Let U_d be the discretized AIS and let $X_0 = \{x_0\}$. Then

$$X_{k+1} = \{\Phi_{\Delta t}(x, u) : x \in X_k, u \in U_d\}, \quad (6)$$

$$Y_k = \{g(x) : x \in X_k\}. \quad (7)$$

Remove trajectories that violate the process constraints under the chosen admissibility definition. If constraints apply continuously, endpoint checks alone are insufficient; intermediate accepted states or simulator event checks are needed.

Every child state needs a reproducible state reference and a link to its parent and input. This can be implemented with validated snapshots or by restoring the base case and replaying the entire stored input sequence. Snapshot reuse requires checking full state and DLL consistency; replay requires deterministic timing and command application and can be computationally expensive.

```

1 frontier = [base_state_reference]
2 for k in range(number_of_intervals):
3     children = []
4     for parent in frontier:
5         for u in input_grid:
6             restore_complete_state(parent)
7             synchronize_bridge_with_restored_state()
8             apply_command_and_advance(u, delta_t)
9             child = validate_and_capture_state(parent, u, k + 1)
10            if child.is_admissible:
11                children.append(child)
12        save_state_nodes_and_outputs(k + 1, children)
13    frontier = children # Exhaustive baseline for a small horizon.

```

Listing 8: Conceptual propagation algorithm; not a package API

With 25 input combinations, there are 25^k possible input sequences after k intervals before merging or pruning. Start with a short exhaustive calculation to validate state propagation. The simulation lifecycle remains under the APS adapter; region analysis can then use the supported Opyrability representation or a documented geometry conversion.

10.6 Step 6: Evaluate Boundary-Based Reduction

The article's exterior-node strategy aims to reduce the number of states propagated. For the four-tank implementation, keep the full stored state associated with every candidate output point before applying a geometric selection rule.

An alpha shape describes the geometry of sampled outputs. It does not establish that interior projected points are irrelevant to future reachability. Different hidden states can share the same (h_1, h_2) coordinates. Consequently, pruning by a two-dimensional boundary must be treated as a tested approximation unless additional model properties justify it.

Compare the reduced method with the short exhaustive baseline using region coverage, missing reachable points, output extrema and computational cost. Record the alpha-shape parameters and any coordinate normalization. If the selected geometry is nonconvex, compute its own area; replacing it with a convex hull can fill unreachable gaps.

10.7 Step 7: Report Results and Connect to Closed-Loop Work

For each horizon or interval, save the evaluated points, state references, construction settings, region area, DOS intersection, OI convention and runtime. Keep the DOS fixed when comparing time-dependent coverage. Report whether the sets describe reachability at exactly time t_k or reachability at any time up to t_k ; the latter is a union of sets and answers a different question.

A nearly constant area alone does not prove convergence of the reachable set or that all trajectories have reached steady state. Check geometry, output extrema, state behavior and sensitivity to the input and time discretization.

For later controller studies, use UDP to exchange measurements and commands at defined simulation-time instants, and use the APS lifecycle interface to coordinate the intervals. Document the controller state and reset it alongside the process. Open-loop reachable coverage describes

process potential under the allowed inputs; the performance of a particular controller is a separate result.

STEP Recommended progression for the four-tank project

Verify the required Opyrability installation; validate the fixed-horizon adapter; reproduce a small AIS-to-output map; define the level DOS; evaluate and refine its coverage; implement complete-state branching; compare exhaustive and boundary-reduced propagation; then extend the verified workflow to closed-loop experiments.